Project on effective dimension for quantum neural networks

Abbas et al., "The power of quantum neural networks", Nature Computational Science 1, 403-409 (2021).

The goal of this project is to reproduce selected results from the target paper using modern Qiskit. The project focuses on effective dimension, Fisher information spectrum and comparison between quantum neural networks and classical neural-network baselines.

Reference implementation and starting material: the original code repository is https://github.com/amyami187/effective_dimension. The modern Qiskit tutorial to use as the starting notebook is Effective Dimension of Qiskit Neural Networks. The arXiv version of the paper is https://arxiv.org/abs/2011.00027.

Questions

1. How do QNNs compare with classical feedforward neural networks at comparable trainable-parameter counts?
2. How does the Fisher information spectrum differ between classical neural networks, easy quantum models, and more expressive QNNs?
3. Does the local effective dimension after training differ significantly from the global effective dimension before training?
4. Do Fisher-spectrum properties correlate with trainability, loss convergence, and generalization?
5. How sensitive are the results to the number of input samples, weight samples, random seeds, feature maps, and ansatz depth?

Task 1 - Build the dataset module

- Implement synthetic binary classification datasets using sklearn.datasets.make_classification and, optionally, make_moons or make_circles.
- Implement Iris preprocessing using sklearn.datasets.load_iris. Required binary tasks: setosa vs versicolor and versicolor vs virginica. Optional: full three-class one-vs-rest classification.
- Standardize all features and rescale them to ranges suitable for Qiskit feature maps, for example [-pi, pi] or [0, pi].
- For higher-dimensional experiments, include PCA-reduced variants with 2 and 4 features.

Task 2 - Reproduce the Qiskit effective-dimension tutorial

- Start from the Qiskit Machine Learning tutorial "Effective Dimension of Qiskit Neural Networks".
- Implement the basic SamplerQNN example using a feature map, ansatz, parity interpretation, EffectiveDimension, and LocalEffectiveDimension.
- Verify that the code computes effective dimension for an array of dataset sizes n.
- Document the difference between global effective dimension and local effective dimension.

Task 3 - Implement quantum neural-network architectures

- Implement a modular QNN constructor in Qiskit. The constructor should accept number of qubits, feature map type, ansatz type, ansatz depth, and output interpretation.
- Easy quantum model: ZFeatureMap plus RealAmplitudes, shallow feature-map depth.
- QNN: ZZFeatureMap or another entangling feature map plus RealAmplitudes or EfficientSU2.
- Test at least 2-qubit and 4-qubit models.
- For each architecture, report number of trainable parameters, number of input parameters, circuit depth, and two-qubit gate count.
- Implement model outputs using SamplerQNN or EstimatorQNN. Use SamplerQNN for parity-style classification and EstimatorQNN for expectation-value outputs if useful.

Task 4 - Implement classical baselines

- Implement classical feedforward neural networks using Keras or other libraries (PyTorch or scikit-learn).
- Use architectures with trainable-parameter counts comparable to the QNNs whenever possible.
- Report train accuracy, test accuracy, loss curves, number of parameters, and runtime.

Task 5 - Compute effective dimension and reproduce the main comparison plot

- Use Qiskit Machine Learning EffectiveDimension for the QNNs.
- For classical models, either implement the Fisher-information calculation directly or adapt the logic of the original paper code.
- Compute normalized effective dimension d_eff/d for at least three models: classical NN, easy quantum model, and QNN.
- Reproduce Figure on: **Effective dimension versus dataset size.**

Task 6 - Compute Fisher spectra

- Extract or compute the normalized Fisher information matrix for selected models.
- Plot eigenvalue distributions for the classical NN, easy quantum model, and expressive QNN.
- Reproduce Figure on: **Fisher information eigenvalue spectrum.**
- Compare spectra across input dimensions, for example 2 and 4 features, or across 2 and 4 qubits.
- Discuss the connection to trainability and barren plateaus

Task 7 - Train the models and compute local effective dimension

- Train selected QNNs and classical baselines on one synthetic dataset and the Iris binary tasks.
- Use several random initializations, for example 5 seeds for the reduced project and 20 seeds if runtime permits.
- For QNNs, use COBYLA, SPSA, SLSQP, or another optimizer supported by Qiskit Machine Learning.
- Plot training loss and test accuracy versus iteration; reproduce Figure on: **Training loss and test accuracy** After training, compute local effective dimension at the trained parameters. reproduce Figure on: **Local versus global effective dimension**